Web UI Design (sdr_web, pocket_web)

Date: 2026-08-08

Summary

This note describes the design of the PocketSDR Web UI. The receiver AP `pocket_web` embeds a small HTTP + WebSocket server implemented in `src/sdr_web.c` (part of libsdr). The Web UI itself is a static single-page application (plain JavaScript + CSS, no framework, no build step) served by that server. The browser communicates with the AP over a single WebSocket connection: JSON text messages for commands and low-rate status, binary frames for high-rate bulk data (PSD, correlator snapshot, correlator history). PSD and correlator history views update at 10 Hz.

The goal is functional parity with the desktop GUI `python/pocket_sdr.py`, monitoring and receiver control alike.

The feature was first developed inside `pocket_trk` and split off as the separate AP `pocket_web` once the receiver lifecycle moved to the browser: the two APs then wanted opposite command lines, `pocket_trk` a full one and the server almost none. `pocket_trk` is back to the CLI receiver it was.

Requirements

1. `pocket_trk` itself hosts the web server and the command / monitor-data exchange for the Web UI.
2. The Web UI is JS + CSS. Transport is WebSocket; bulk data uses binary frames.
3. The UI eventually reproduces `python/pocket_sdr.py`; the first version may provide only basic screens.
4. The PSD and correlator-history screens update at 10 Hz.
5. The server-side UI support is new code in `src/sdr_web.c`.

Requirement 1 is met by `pocket_web` rather than by `pocket_trk`; see the Summary.

Goals and Non-Goals

Goals:

- Zero new external dependencies. HTTP/1.1 (static files only) and the RFC 6455 WebSocket subset needed here are small enough to implement directly on BSD sockets / winsock2, in C99, matching the existing code style. SHA-1 and Base64 (WebSocket handshake only) are implemented as small static functions in `sdr_web.c`.
- Reuse the existing monitor/control API of `src/sdr_rcv.c` unchanged. The Python GUI already drives the receiver exclusively through `sdr_rcv_ch_stat()`, `sdr_rcv_corr_stat()`,

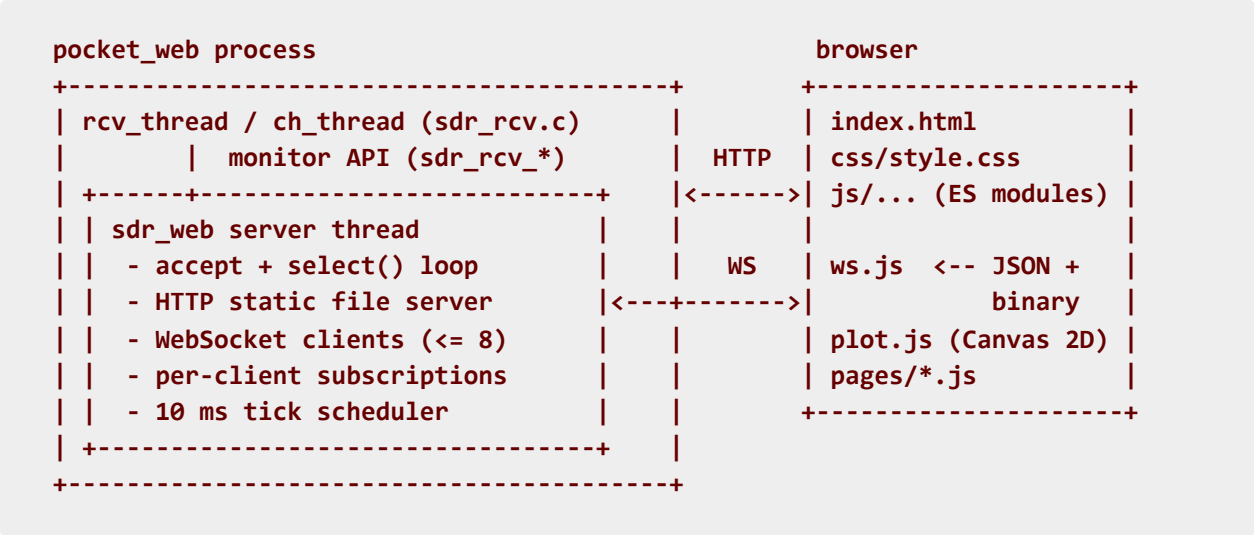
`sdr_rcv_rfch_psd()` etc.; `sdr_web.c` only adds a transport + encoding layer on top of the same calls. No public API changes.

- Keep `pocket_trk` free of the Web UI. The server AP is separate, so the CLI receiver keeps its command line and its behavior.
- Receiver lifecycle from the browser: `pocket_web` starts with the receiver stopped and the browser configures, starts and stops it. The settings of the last session are restored at start and saved at exit.

Non-Goals (for now, see Future Extensions):

- Authentication and TLS. The server binds to 127.0.0.1 by default; remote use should go through a reverse proxy.

Architecture



One server thread handles everything (accept, HTTP, WebSocket I/O, periodic sampling and push). All receiver access goes through the existing thread-safe monitor API, so no new locking is required in the receiver core.

Server Design (src/sdr_web.c)

Public API

```c
/* pocket_sdr.h */
typedef struct sdr_web_tag sdr_web_t;

sdr_web_t *sdr_web_start(sdr_rcv_t *rcv, const char *addr, int port,
    const char *html_dir);
void sdr_web_init_cfg(sdr_web_cfg_t *cfg);
void sdr_web_set_cfg(sdr_web_t *web, const sdr_web_cfg_t *cfg,
    const char *file);
int sdr_web_load_cfg(sdr_web_t *web);
int sdr_web_start_rcv(sdr_web_t *web);
sdr_rcv_t *sdr_web_stop(sdr_web_t *web);
```

- `sdr_web_start()` binds `addr:port` (TCP, `SO_REUSEADDR`), starts the server thread (`sdr_thread_create()`), and returns an opaque handle. NULL on error. `rcv` may be NULL: the server then starts with no receiver.
- `sdr_web_init_cfg()` fills a `sdr_web_cfg_t` with the defaults used on the first start without a settings file.
- `sdr_web_set_cfg()` hands the configuration to the server and enables the lifecycle and configuration commands. Without it the server is monitor-only. `file` is the settings file, saved when the receiver stops and when the server stops.
- `sdr_web_load_cfg()` restores the configuration and the system options from that file.
- `sdr_web_start_rcv()` starts the receiver from the stored configuration, as the Start command does, for the `-start` option of `pocket_web`.
- `sdr_web_stop()` signals the thread, closes all sockets, joins and frees. The current receiver is not closed; it is returned to the caller.
- Winsock is initialized by `sdr_func_init()` -> `strinitcom()`, which `pocket_web` calls before starting the server, and `-lws2_32` / `-lwsock32` are already linked; no build-level changes beyond adding the object file.

Thread model

Single server thread, `select()` based event loop with a 10 ms timeout that doubles as the scheduler tick:

1. `select()` on the listen socket + client sockets (read set; write set only for clients with pending output).
2. Accept new connections (up to `MAX_WEB_CLI = 8`, else 503/close).
3. Read and parse per client: HTTP request until upgrade, WebSocket frames after upgrade.
4. On each tick, for every (client, topic) subscription whose period elapsed (`sdr_get_tick()`), sample the receiver, encode, and send.

Per-client threads are avoided deliberately: client count is small, payloads are small, and a single thread makes the subscription state trivially consistent.

HTTP server

Static file serving only, enough for the UI to be self-hosted:

- `GET` only; anything else 405. Request line + headers capped at 4 KB.
- `/` maps to `index.html`. Paths are resolved under `html_dir`; any path containing `..` is rejected (403).
- Content types by extension: html, css, js, mjs, png, svg, ico, json, woff2.
- `Connection: close` after each response (the only long-lived connection is the WebSocket). No caching headers in v1 (dev-friendly).
- Default `html_dir` is `<exe_dir>/../html` (exe dir via `GetModuleFileName` / `/proc/self/exe` / `_NSGetExecutablePath`), falling back to `./html`; overridable by the `-html` option.

WebSocket implementation

Endpoint `GET /ws` with `Upgrade: websocket`. RFC 6455 subset:

- Handshake: `Sec-WebSocket-Accept = base64(sha1(key + RFC6455-GUID))`.
- Opcodes: 0x1 text, 0x2 binary, 0x8 close, 0x9 ping, 0xA pong. Client frames are masked (required), server frames unmasked.
- Client-to-server messages are commands only and are capped at 4 KB (fragmented messages are assembled up to that cap, then the client is dropped). Server-to-client frames are never fragmented.
- Keepalive: server pings every 10 s, drops a client after 30 s without pong.

Subscription model and scheduler

Each client holds a small table of active subscriptions:

```
typedef struct {
    int topic;              /* WEB_TOPIC_??? */
    int cyc;                /* update cycle (ms), clamped to >= 50 */
    uint32_t next;          /* next due tick (ms) */
    double prm[4];          /* topic parameters (tave, tspan, width, ...) */
    int ch, rfch;           /* target channels */
    char sys[16], sats[256];/* text parameters */
} web_sub_t;
```

- `sub` creates or replaces the subscription for that topic (re-`sub` with new parameters is the way to change them).
- `unsub` removes it. All subscriptions die with the connection.
- `get` performs a one-shot sample of a topic without subscribing.
- The scheduler skips (does not queue) a topic push while more than 32 KB is still pending for that client: latest-data-wins, no unbounded queues. Send is non-blocking with a 256 KB per-client output buffer; a client that stays blocked past that is dropped.

Monitor topics

Topics map 1:1 onto existing monitor calls. Text-producing APIs are passed through verbatim inside JSON; the JS side ports the column parsing that `pocket_sdr.py` already does. This keeps the C side thin, at the cost of making the existing status formats part of the wire protocol (acceptable — they are already the de-facto GUI API).

topic	source API	payload	default cyc
`rcv_stat`	`sdr_rcv_rcv_stat()` + `sdr_rcv_str_stat()` + CPU load	JSON	200 ms
`ch_stat`	`sdr_rcv_ch_stat()` (sys/chno/min_lock/rfch/opt from sub)	JSON	200 ms
`sat_stat`	`sdr_rcv_sat_stat()` per sat in `sats` param	JSON	200 ms
`pvt_sol`	`sdr_rcv_pvt_sol()`	JSON	200 ms
`rfch_stat`	`sdr_rcv_rfch_stat()` all RF+array CHs	JSON	200 ms
`hist`	`sdr_rcv_rfch_hist()`	JSON	200 ms
`log`	`sdr_get_log()` (server-side fan-out)	JSON	200 ms
`array_stat`	`sdr_rcv_array_stat()` + beams + ant_pos	JSON	500 ms
`opts`	current `sdr_rcv_setopt()` option values	JSON	200 ms
`cfg`	receiver configuration (`sdr_web_cfg_t`)	JSON	on get
`psd`	`sdr_rcv_rfch_psd()` (rfch 0: all RF CHs)	binary 1	100 ms
`corr`	`sdr_rcv_corr_stat()`	binary 2	100 ms
`corr_hist`	`sdr_rcv_corr_hist()`	binary 3	100 ms

Notes:

- `ch_stat` needs a large buffer: 128 B x `SDR_MAX_NCH` (1500) = 192 KB, allocated once in `sdr_web_t` (`sdr_malloc()`), same sizing as the Python GUI, not the small static buffer the console status display of `pocket_trk` uses.
- `rcv_stat` carries the process CPU load in `cpu`, measured by the server thread once a second from the process CPU time (`GetProcessTimes()` on Windows, `getrusage()` elsewhere) over the elapsed wall time, divided by the number of cores. 100 % therefore means every core is saturated, so the value answers "does this machine keep up?" rather than "how busy is one core?". It is measured once and shared, so all clients see the same value.
- `rcv_stat` also carries the per-stream status in `strs` (-1: error, 0: close, 1: wait, 2: connect, 3: active) as the Receiver page LEDs. -1 covers an output stream that failed to open, which stays in the slot as a status carrier without a stream (`sdr_rcv_str_stat()`).
- `psd` samples `sdr_rcv_rfch_psd(rcv, ch, tave, nfft, psd)` with `tave` and `nfft` (default 2048) from the subscription; returns N/2 bins for I sampling, N for IQ.
- `corr_hist` samples `sdr_rcv_corr_hist(rcv, ch, tspan, stat, P)` ; n = min(tspan/T, 5000) points.
- 10 Hz for `psd` / `corr` / `corr_hist` satisfies requirement (4); `cyc` is clamped to >= 50 ms so a client cannot exceed the Python GUI's UD_CYCLE1.

- The `default cyc` column is what the Web UI subscribes with. A `sub` that omits `cyc` falls back to 100 ms for the JSON topics and 50 ms for the binary ones.

Commands

cmd	arguments	API
`sel_ch`	`ch`, `width` (s)	`sdr_rcv_sel_ch()`
`set_gain`	`rfch`, `gain`	`sdr_rcv_set_gain()`
`get_gain`	`rfch`	`sdr_rcv_get_gain()`
`set_filt`	`rfch`, `bw`, `freq`, `order`	`sdr_rcv_set_filt()`
`get_filt`	`rfch`	`sdr_rcv_get_filt()`
`setopt`	`name`, `value`	`sdr_rcv_setopt()`
`log_level`	`value`	`sdr_log_level()`
`array_run`	`run` (1:start, 0:stop, 2:clear)	`sdr_rcv_array_run()`
`array_mode`	`mode` (0:both, 1:bias, 2:rpy)	`sdr_rcv_array_set_mode()`
`array_beam`	`rfch`, `az`, `el` (deg)	`sdr_rcv_array_set_beam()`
`array_save`	`file` (default array_calib.txt)	`sdr_rcv_array_save()`
`array_load`	`file` (default array_calib.txt)	`sdr_rcv_array_load()`

Receiver lifecycle commands (enabled when the AP passes its configuration via `sdr_web_set_cfg()`; the flat values mirror the `sdr_web_cfg_t` fields, with `fs` / `fo` / `lpf` in MHz and multi-value fields as comma-separated strings, output paths separated by `|`):

cmd	arguments	action
`start`	-	open the receiver from the stored configuration
`stop`	-	close the running receiver (Web UI stays up)
`set_inp`	`inp`, `file`, `fmt`, `fs`, `fo`, `IQ`, `bits`, `lpf`, `toff`, `tscale`, `bus`, `port`, `conf`, `conf_ena`, `dev_opt`, `driver`	update input configuration
`set_sig`	`sigs` ("SIG:prns ..."), `satno`, `rfch`	update signal configuration
`set_out`	`types`, `paths`, `log_mask`, `array_sep`	update output streams
`set_sys`	`fftw`, `opt`, `fast_acq`	update system configuration
`opts_default`	-	restore the default system options

`set_*` are rejected while the receiver runs. On `start` / `stop` the server broadcasts an updated `hello` (with `run` and `cfg_ena` flags) to all clients. The AP stays up when a file input ends or when an open

fails; it goes idle until started again from the Web UI.

Options that the receiver takes as tokens of its options string but the UI presents as their own controls -- the digital LPF bandwidths, the fast acquisition mode and the RF CH separation -- are configuration fields of their own (`lpf_bw[]` , `fast_acq` , `array_sep`) and are composed into the options string when the receiver is opened. The Receiver Options text stays as the user typed it instead of gaining and losing `-LPF=` / `-FAST_SRCH` / `-ARRAY` tokens behind their back. The RF frontend device options (`-GAIN=` / `-BW=`) are a separate field (`dev_opt`) as in the Tk GUI, and the device configuration file has an enable flag (`conf_ena`) so that its path survives being switched off.

The satellite number lists of the Signal page are a configuration field of their own (`satno[]` , one `|` separated entry per satellite system in the order GPS, GLONASS, Galileo, QZSS, BeiDou, NavIC, SBAS) as in the Tk GUI. They are the user setting the signal PRN lists are generated from, not the other way around: the QZSS number-to-PRN and the GLONASS FCN/slot mapping depend on the signal, so a list reconstructed from `sigs` would change with the signals selected. Satellite numbers a signal has no PRN for are dropped when the receiver channels are created.

Every command is answered with an `ack` JSON message. `setopt` accepts only the key names already handled by `sdr_rcv_setopt()` .

Correlator-channel selection policy

`sdr_rcv_sel_ch()` enables the `SDR_N_CORRX` (101) extra correlators on exactly one channel and disables them elsewhere, so it is a global, contended resource:

- The server owns a single "selected channel" state; a `corr` / `corr_hist` subscription (or explicit `sel_ch` command) sets it, last-writer-wins.
- Every change is broadcast to all clients as `{"type":"sel_ch",...}` so their Correlator pages can follow.
- When no client subscribes `corr` / `corr_hist` , the server calls `sdr_rcv_sel_ch(rcv, 0, 3e-6)` to disable the extra correlators — the same load-shedding the Python GUI performs when the Correlator page is hidden.

Log topic

`sdr_get_log()` is a destructive read (the 512 KB accumulation buffer is cleared on read), so it must have a single consumer. The server thread polls it every 200 ms into an internal ring of 2000 lines (like the GUI's `MAX_RCVLOG`) and pushes the increments to all subscribed clients. The `-log` output streams are unaffected (they are fed via `sdr_log_add_str()` , a separate path). A newly connected client receives the current ring content once, then increments.

Locking and thread safety

- All receiver reads go through the monitor API, which either takes the appropriate mutex internally (`sdr_ch_corr_stat/hist` -> `ch->mtx`, `sdr_pvt_solstr` -> `pvt->mtx`) or does lock-free approximate reads (`sdr_rcv_ch_stat`), exactly as the Python GUI does today.
- Command APIs (`sdr_rcv_setopt`, `sdr_rcv_sel_ch`, gain/filter) are already called cross-thread by the GUI; same usage here.
- `sdr_web_t` state (clients, subscriptions, log ring) is owned by the server thread only; `sdr_web_stop()` synchronizes via a stop flag + thread join.

Wire Protocol

Protocol version 1. All JSON is UTF-8 in WebSocket text frames; all binary frames are little-endian (both supported hosts are LE).

Session start

On WebSocket open the server sends:

```
{"type":"hello","name":"Pocket SDR","ver":"0.20","proto":1,
 "nrfch":2,"narch":0,"nch":121,"fs":24000000.0,"sel_ch":0,
 "run":1,"cfg_ena":1}
```

Client to server (JSON)

```
{"cmd":"sub","topic":"psd","rfch":1,"cyc":100,"tave":0.01,"nfft":2048}
{"cmd":"sub","topic":"corr","ch":12,"cyc":100,"width":5e-6}
{"cmd":"sub","topic":"corr_hist","ch":12,"cyc":100,"tspan":1.0}
{"cmd":"sub","topic":"ch_stat","cyc":200,"sys":"ALL","chno":0,
 "min_lock":2.0,"rfch":0,"opt":1}
{"cmd":"sub","topic":"sat_stat","cyc":200,"sats":"G01,G05,J01"}
{"cmd":"unsub","topic":"psd"}
{"cmd":"get","topic":"rcv_stat"}
{"cmd":"sel_ch","ch":12,"width":5e-6}
{"cmd":"set_gain","rfch":1,"gain":-1}
{"cmd":"setopt","name":"el_mask","value":15.0}
```

Values are flat (string / number / bool); no nested objects or arrays are required, which keeps the embedded JSON parser minimal (~100 lines).

Server to client (JSON)

```
{"type":"rcv_stat","str":"...one line...","strs":[3,0,0,0,0,0,0,0],
 "cpu":12.3}
{"type":"ch_stat","str":"header\ncolumns\nCH lines..."}
{"type":"pvt_sol","str":"2026-08-01 12:34:56.0 ..."}
{"type":"sat_stat","sats":[{"sat":"G01","az":123.4,"el":45.6,"pvt":1,
 "obs":1,"eph":1,"svh":0,"fcn":0}]}
{"type":"rfch_stat","chs":[{"ch":1,"dev":1,"fmt":2,"fs":24.0,"fo":1568.0,
 "IQ":2,"bits":2,"std":9.9,"rtoc":0}]}
{"type":"hist","rfch":1,"val":[-3,-1,1,3],"hist1":[...],"hist2":[...]}
{"type":"log","lines":["$POS,...","$CH,..."]}
{"type":"sel_ch","ch":12,"width":5e-6}
{"type":"ack","cmd":"set_gain","ok":true,"msg":""}
{"type":"error","msg":"..."}
```

Binary frames

Common rule: byte 0 is the frame type. JS decodes with `DataView` / `Float32Array`.

Type 1 — PSD (topic `psd`):

offset	type	field
0	u8	frame type = 1
1	u8	RF CH (1-origin)
2	u8	IQ (1:I, 2:IQ)
3	u8	bits
4	f32	fs (Hz)
8	f32	tave (s)
12	u32	n (PSD bins)
16	f64	fo (Hz)
24	f32[n]	PSD (dB/Hz); I: [fo, fo+fs/2], IQ: [fo-fs/2, fo+fs/2], DC centered

Type 2 — correlator snapshot (topic `corr`):

offset	type	field
0	u8	frame type = 2
1	u8	state (1:IDLE, 2:SRCH, 3:LOCK)
2	u16	BB CH number
4	u32	n (total correlators = npos + nposx, <= 107)
8	u32	npos (standard: [0]=P, [1]=E, [2]=L, [3]=N, [4]=VE, [5]=VL)
12	f32	fs (Hz)
16	f32	lock time (s)
20	f32	C/N0 (dB-Hz)
24	f32	Doppler fd (Hz)
28	f32	receiver time (s)
32	f64	code offset coff (ms)

offset	type	field
40	f32[n]	correlator positions pos (samples)
40+4n	f32[2n]	instantaneous C, I/Q interleaved
40+12n	f32[n]	aveP (average correlation power)
40+16n	f32[n]	aveI (average I*sign(IP))

Size 40 + 20n; n = 107 -> 2180 B.

Type 3 — P-correlator history (topic `corr_hist`):

offset	type	field
0	u8	frame type = 3
1	u8	reserved
2	u16	BB CH number
4	u32	n (points, <= 5000)
8	f64	receiver time of newest point (s)
16	f32	T, integration time per point (s)
20	u32	reserved
24	f32[2n]	P history, I/Q interleaved, oldest first

Size 24 + 8n; tspan = 1 s at T = 1 ms -> 1000 points -> 8 KB.

Web UI App (html/)

Files

```
html/
  index.html
  css/style.css
  js/main.js         app shell: tabs, button bar, status bar
  js/ws.js           WebSocket client: reconnect, sub/unsub bookkeeping,
                     JSON dispatch, binary frame decode
  js/plot.js         Canvas-2D plot library (port of python/sdr_plot.py:
                     axes, 1-2-5 ticks, time axis, sky plot, markers)
  js/pages/rcv.js    Receiver page
  js/pages/rfch.js   RF CH page (PSD + histograms)
  js/pages/bbch.js   BB CH page (channel table)
  js/pages/corr.js   Correlator page
  js/pages/sats.js   Satellites page
  js/pages/sol.js    Solution page
  js/pages/array.js  Array page
  js/pages/log.js    Log page
  js/pages/inp.js    Input Options page
  js/pages/out.js    Output Options page
  js/pages/sig.js    Signal Options page
  js/pages/opts.js   System Options page
  js/pages/help.js   Help page
  img/favicon.svg
```

The Options and Help pages have no tab; they are reached from the command buttons in the title bar (Start, Stop, Input, Output, Signal, System, ?).

Plain ES modules, no framework, no bundler — the server serves the tree as-is, matching the project's no-dependency policy. Layout uses CSS grid/flexbox (the Tk `place(relx/relwidth)` layouts translate directly).

Rendering

- Plots are Canvas 2D, one canvas per plot panel, redrawn on data arrival. 10 Hz redraw of the three Correlator panels plus one PSD panel is far below Canvas limits; no requestAnimationFrame accumulation is needed, but a "latest frame only" guard drops stale frames if the tab is throttled.
- Only the visible page holds its subscriptions: switching tabs unsubscribes the old page's topics and subscribes the new page's (mirrors `pages_update()` updating only the current tab, and drives the sel_ch load-shedding policy server-side).
- Reconnect with exponential backoff; on reconnect the current page re-subscribes.

Pages and phases

All four phases below are implemented.

Phase 1:

- **Shell**: title bar with receiver name/version (from `hello`), command buttons and a connection indicator that blinks at 1 Hz while the receiver runs (`run` of `hello`), tab bar, status bar (message + `CPU: xx.x %  Time: xxx s` from `rcv_stat`).
- **Receiver**: text status table (2 columns x 11 rows, same labels as `update_rcv_stat()`), output-stream LEDs, PVT solution line. No sky plot / C/N0 bar yet.
- **BB CH**: channel table from `ch_stat` (16 short-format columns, C/N0 bar drawn client-side 25-50 dB-Hz, idle/search row coloring, row click selects the Correlator channel).
- **RF CH**: single-CH view: PSD plot (10 Hz, Y fixed [-85, -45] dB/Hz, LO marker, signal frequency marks) + I/Q histograms (`hist` topic), Ave(s) selector; gain/filter selectors wired to `set_gain` / `set_filt`.
- **Correlator**: the three panels of `corr_page` at 10 Hz — correlator shape plot (I*sign(IP) etc. with IQ/W/T/Range selectors), IP-QP scatter (history), IP/QP time series with status text overlay; CH prev/next and selection synced via `sel_ch` broadcasts.
- **Log**: filter combo + free text (AND = space, OR = `|`), pause/clear, monospace scrollback (2000 lines).

Phase 2: Receiver page sky plot + signal C/N0 bar chart, RF CH "ALL" frequency-band map and 2x2 PSD tiling, Satellites page, Solution page (client-side solution ring like `sol_log`, Pos ENU / Horiz modes).

Phase 3: Array page (`array_*` topics/commands), System Options page (`setopt`), gain heatmap overlay + beam setting on the sky plot.

Phase 4: receiver lifecycle from the browser — Start / Stop and the Input, Output and Signal Options pages, laid out after the Tk dialogs, with the settings saved to and restored from the AP settings file. This is what made the server its own AP; see the Summary.

Responsive layout: the pages reflow below 620 px so a phone can drive the receiver — the Receiver page stacks status, sky plot and C/N0 in thirds, the RF CH page puts the PSD over the histograms, and the Correlator page stacks its three panels. The status bar shows the CPU load and the receiver time on their own line above the message there; in the single-line layout a message too long for the bar is cut with an ellipsis instead of pushing them out.

Touch navigation: a left/right swipe on the page area steps to the next or previous tab page (Receiver ... Log; the pages of the command bar are not in the sequence and do not swipe), with a short slide-in as the feedback, and the tab bar scrolls the selected tab into view. The tables of the BB CH, Satellites and Log pages are wider than a phone screen, so a gesture over them is shared: a short drag scrolls the table sideways as usual, and only a swipe longer than 40 % of the page width (200 px at most) switches the page, which then puts the table back where the swipe took it from. Everywhere else 40 px is enough.

Visual style

Ported from the Tk GUI so both frontends look alike:

- Page background `#F0F0F0` / plot background `#FFFFFF`, frame and tick `#555555`, grid `#CCCCCC`, primary plot `#003020`, secondary `#888844`, reference `#BBBBBB`, warning `#FF4000`.

- Satellite system colors: G `#006600`, R `#EE9900`, E `#CC00CC`, J `#0000AA`, C `#CC0000`, I `#007777`, S `#777777` (+ pale variants).
- Fonts: `"DejaVu Sans", "Noto Sans", sans-serif` at 10px, 9px for tables and plot labels; tables and log in `"DejaVu Sans Mono", "Noto Sans Mono", monospace`. Numeric cells right-aligned.
- The theme is a set of `:root` custom properties in `css/style.css`; the plot library takes its fonts from there through `cssVar()` and keeps its colors as its own constants.
- Base layout 800x600, fully responsive via flex/grid.

pocket_web AP

`app/pocket_web/pocket_web.c` is the whole AP, about 150 lines. It needs no receiver-open code of its own: `sdr_web.c` builds the receiver from the configuration.

```
pocket_web [-web [addr:]port] [-html dir] [-ini file] [-start]
           [-debug file] [-v]
```

```
sdr_func_init("");                /* also initializes the socket library */
web = sdr_web_start(NULL, addr, port, html_dir);
sdr_web_init_cfg(&cfg);
sdr_web_set_cfg(web, &cfg, ini_file);
sdr_web_load_cfg(web);            /* settings of the last session */
if (start) sdr_web_start_rcv(web); /* -start */
while (!intr) sdr_sleep_msec(100); /* lifecycle belongs to the browser */
rcv = sdr_web_stop(web);
if (rcv) sdr_rcv_close(rcv);
```

The AP prints no runtime status of its own; the receiver is monitored from the Web UI. It stays up when the receiver fails to open, so the settings can be fixed from the browser.

Shutdown matters here, because a session that dies without closing the device leaves the RF frontend streaming. On Windows the C runtime raises SIGINT only for Ctrl-C and Ctrl-Break, while a console close, logoff or shutdown terminates the process as soon as the handler returns, so the AP installs a `SetConsoleCtrlHandler()` that waits for the main thread to close the device (`pocket_trk` does the same). `run_sdr.sh` starts and stops the server; on MSYS2 it stops it with SIGQUIT, which reaches a native Windows process as CTRL_BREAK_EVENT, because plain `kill` is `TerminateProcess()` and would leave the frontend streaming.

`doc/command_ref.md` documents the AP; `doc/api_ref.md` documents the `sdr_web.c` API.

Build Changes

- `lib/build/libsdr.mk`: add `sdr_web.o` to `OBJ`, plus its compile rule and the `pocket_sdr.h` dependency line (3 places).
- `src/pocket_sdr.h`: `sdr_web_t` opaque typedef, `sdr_web_cfg_t` and the prototypes.
- `app/pocket_web/makefile`: new, a copy of the `pocket_trk` one with a single target (links `libsdr.a`; `-lws2_32` is already present on Windows).
- `test/utest/makefile`: `test_sdr_web` exercises the server over a real loopback socket with a minimal HTTP/WS client — start/stop, static serving (200 / 404 / path traversal), the RFC 6455 handshake and `hello`, commands and `ack`, the configuration defaults and their round trip through the settings file, and `sdr_web_start_rcv()` — avoiding test-only hooks in the source.

Performance Estimates

Per client, defaults (fs = 24 Msps, 100-CH receiver, tspan = 1 s):

topic	payload	rate	bandwidth
`psd`	~8.2 KB	10 Hz	~82 KB/s
`corr`	~2.2 KB	10 Hz	~22 KB/s
`corr_hist`	~8 KB	10 Hz	~80 KB/s
`ch_stat`	~11 KB	5 Hz	~55 KB/s
others	< 2 KB	5 Hz	~10 KB/s

Roughly 0.25 MB/s per fully-subscribed client; 8 clients ~2 MB/s — no issue on localhost or LAN.

CPU: one `sdr_rcv_rfch_psd()` call at tave = 0.01 copies 240 k samples and runs ~117 x 2048-point FFTs; at 10 Hz this is a few ms/s of one core. `corr_hist` is a 40 KB memcpy worst case. The extra-correlator cost is bounded by the same sel_ch policy the Python GUI uses. Encoding is memcpy-level. Total server overhead is negligible next to the receiver itself, and the single-thread receive path requirement is untouched (the web server runs on its own thread and only reads).

Security Considerations

- Binds 127.0.0.1 when `-web` gives a port alone, so exposing the interface is an explicit act (`-web 0.0.0.0:port`). No warning is printed for it.
- No TLS/auth in scope; for remote access, front with a reverse proxy (nginx/caddy) providing HTTPS + auth. WSS works transparently through such a proxy.
- Path traversal blocked in the static file server; request size capped; client count capped; per-client output bounded (slow clients dropped).
- Commands can change receiver behavior (`setopt`, gain, filter) — another reason the default bind is loopback-only.

Future Extensions

- Embedding the `html/` tree into the binary (generated C arrays) for a single-file distribution; `-html` would then switch to disk for development.
- Array control page and calibration monitor.
- Optional `Origin` check on the WebSocket handshake.

Implementation Milestones

1. `sdr_web.c` core: socket loop, HTTP static serving, WebSocket handshake + framing, JSON command parser, `hello` / `ack` (verifiable with a browser console or `wscat` alone).
2. JSON monitor topics (`rcv_stat`, `ch_stat`, `pvt_sol`, `sat_stat`, `rfch_stat`, `hist`, `log`) + scheduler.
3. Binary topics (`psd`, `corr`, `corr_hist`) + sel_ch policy.
4. `html/` phase-1 UI: shell, plot.js, Receiver / BB CH / RF CH / Correlator / Log pages.
5. Commands (`set_gain`, `set_filt`, `setopt`, `log_level`), AP options, `doc/command_ref.md` update.
6. Phase-2 pages (sky plot, C/N0 bars, band map, Satellites, Solution).
7. Phase-3 Array page and System Options page.
8. Phase-4 receiver lifecycle and the Input / Output / Signal Options pages, settings save and restore.
9. Split the server off as the `pocket_web` AP and revert `pocket_trk`.